ChatGPT

ClearPay: Liquidity-Saving Settlement Research, Novelty Assessment, and AWS Implementation Plan

Executive summary

Your project is not simply "cloud-based netting." The most accurate topic is:

> **ClearPay: A Liquidity-Aware, Fault-Tolerant Payment Clearing and Settlement System Using AWS Serverless Architecture**

The financial problem is straightforward: in an interbank payment queue, some banks may temporarily lack enough settlement liquidity to settle every payment. A naive all-or-nothing batch can therefore delay payments that could have been settled if offsetting obligations were considered together. This is exactly the family of problems studied in the academic literature as **bank clearing, payment gridlock resolution, and liquidity-saving mechanisms (LSMs)**. Güntzer, Jungnickel and Leclerc formalized a Bank Clearing Problem in 1998 in which payment-clearing value is optimized subject to participants' available deposits; subsequent work developed gridlock-resolution algorithms and analyzed LSMs deployed in real RTGS systems. [1]

Real payment systems already use sophisticated forms of this idea. The Bank of England's CHAPS LSM matches queued non-urgent payments for simultaneous offsetting settlement and was reported to reduce intraday liquidity requirements by around 20% after its introduction. TARGET2 used priorities, reservations, queue optimization, bilateral/multilateral offsetting and other event/time-driven LSMs; today's T2 continues to provide priorities, liquidity reservations, limits, pooling and optimization procedures. [2]

Therefore, **you should not claim that partial settlement, bilateral offsetting, gridlock resolution, or group settlement is globally novel**. Your friend's terminal output saying "GENUINE NOVELTY" is too strong. Recent BIS work goes considerably further, including auction-based LSMs where banks express liquidity preferences through bids and side payments, while central-bank research has explored Shapley-value allocation, payment reordering, strategic behavior and even quantum/simulated-annealing optimization. [3]

What **is defensible as project-level novelty** is the combination of:

> **a bounded group-aware liquidity-constrained settlement heuristic + explicit priority/aging policy + deterministic crash-safe execution of the resulting plan through an AWS serverless workflow + visual benchmarking against exact MILP and simple baselines.**

That gives ClearPay a much stronger identity than "we implemented netting."

From the code and screenshots you supplied, the project is currently split into two pieces:

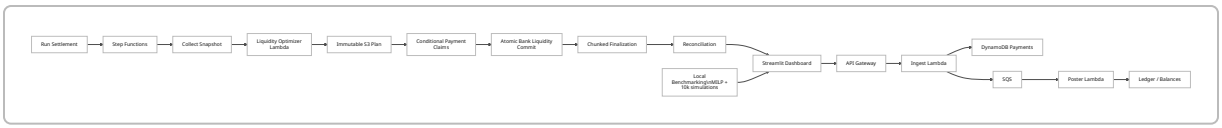
Component	Current state
Your AWS payment-processing backend	Implemented/deployed foundation
API → Lambda → DynamoDB → SQS	Implemented; live `202 PENDING` request demonstrated
Existing netting/settlement/reconciliation handlers	Implemented as foundation
Friend's liquidity selector	Implemented locally
Friend's `all_or_nothing` baseline	Implemented
Friend's cumulative greedy baseline	Implemented
Friend's removal + single add-back + bilateral group add-back	Implemented
Exact SciPy MILP comparator	Implemented locally
Liquidity algorithm tests	44 tests shown passing
Algorithm integrated into the AWS settlement workflow	**Not yet**
Deterministic frozen settlement plan/recovery	**Not yet**
Streamlit dashboard	**Not yet**

Your friend's supplied benchmark currently reports an average **3.97% optimality gap** between the heuristic and MILP over 20 small test instances, and her 200-payment/8-bank simulations show the removal heuristic outperforming the two baselines under tight liquidity. Those are useful **local synthetic results**, not evidence from real banking data or AWS execution.

There is also an important result in her own SDG report: at the tested 50% liquidity setting, the reported liquidity consumed per 1,000 paise settled is **135.69 for the removal heuristic versus 130.32 for all-or-nothing and 132.05 for greedy**. Therefore, your result currently supports **higher settlement completion under liquidity constraints**, not a claim that the heuristic minimizes liquidity per unit settled.

My strongest recommendation is to evolve the current algorithm into a **bounded cycle-aware group-repair heuristic**: keep the current removal approach, but replace the crude “restore every deferred payment between two banks or none” step with bounded search over bilateral subsets and selected three-bank cycles. Add priority aging and optional randomized restarts. Use exact MILP only as a local benchmark or bounded repair tool. This creates a substantially more interesting student project while remaining computationally small enough for local Python and lightweight enough for an AWS Lambda implementation.

The final architecture should look like this:



The large optimization experiments should remain local. AWS should demonstrate the **real transaction lifecycle, orchestration, idempotency and failure recovery**, rather than performing thousands of experimental optimization runs. That is also the safest approach for controlling cloud usage.

Existing research and industry mechanisms

The literature confirms that your underlying financial problem is real, established and considerably broader than simple bilateral netting.

Güntzer, Jungnickel and Leclerc — Bank Clearing Problem. Their 1998 European Journal of Operational Research paper models interbank clearing as a discrete optimization problem in which **clearing volume is the objective and participating banks' deposits are constrained resources**. They propose heuristic approaches and evaluate them through simulation. This is very close conceptually to ClearPay's “choose a feasible subset under per-bank liquidity limits.” It means the basic optimization problem cannot be presented as a new invention. [4]

Shafransky and Doudkin — optimization under limited funds. Their 2006 paper again studies interbank clearing with limited money available to banks, establishes strong computational difficulty for its formulation, and proposes a fast heuristic. This reinforces the rationale for using a heuristic rather than expecting a globally optimal solution at arbitrary scale. [5]

Bech and Soramäki — gridlock resolution. Their Bank of Finland work examines gridlock in real-time interbank payment systems using actual Danish and Finnish RTGS payment data and reports that its gridlock-resolution algorithm reduces queuing, particularly when intraday liquidity is scarce. This is especially relevant to ClearPay because it shifts the framing from “netting” to **resolving settlement gridlock under scarce liquidity**. [6]

CPMI/BIS — evolution from RTGS to liquidity-saving designs. The 2005 CPMI report explains that after the widespread adoption of RTGS for safety and speed, system design increasingly focused on reducing liquidity costs and improving intraday liquidity management. Importantly, the report warns that there is no single universally optimal design because payment systems trade off risks, liquidity costs, delay and participant preferences. [7]

Bank of England — LSM theory. Galbiati and Soramäki model banks deciding both how much liquidity to provide and whether to send payments through an RTGS or LSM stream. They find that LSMs can improve coordination when liquidity is expensive but can still produce second-best outcomes and even undesirable equilibria, because participating banks behave strategically. Jurgilas and Martin show in a collateral-based model that matching and offsetting payments can reduce collateral needs and encourage earlier payment submission. [8]

CHAPS — operational LSM. The Bank of England introduced an LSM into CHAPS settlement infrastructure in 2013. The Bank reported a reduction of roughly 20%, or £4 billion at that time, in banks' intraday liquidity

requirements. Its operational descriptions state that queued non-urgent payments can be matched for simultaneous offsetting settlement. This is an important real-world benchmark for ClearPay: the industry mechanism is about matching queued obligations, not simply executing transactions independently in priority order. [9]

ECB TARGET2/T2 — priorities plus optimization. TARGET2's LSM combined priorities, reservations, timed payments, bilateral and multilateral offsetting and queue optimization. ECB analysis found that richer algorithm configurations reduced unsettled payments and queuing times, although their incremental effect was limited during periods of exceptionally abundant system liquidity. The successor T2 RTGS service continues to provide liquidity-saving features including payment priorities, timed transactions, reservations, limits, liquidity pooling and optimization procedures. [10]

That observation is useful for your experiments: **an LSM should show its strongest benefit under constrained liquidity**, not when every bank is fully funded. This is consistent with the Bank of Finland gridlock results and with your friend's current synthetic benchmarks. [11]

Bank of Canada / Lynx. Research on Canada's transition to Lynx found that its LSM encourages use of the payment queue, enables liquidity pooling and can improve liquidity efficiency, while introducing a modest delay trade-off. Separate research shows that participants can strategically exploit gridlock-resolution arrangements to save liquidity, demonstrating that real participants respond to the mechanism rather than behaving as passive inputs to an optimizer. [12]

BIS cross-country evidence. A 2023 BIS study covering large-value payment systems in nine economies reports average intraday liquidity use around 15% of daily payment values and finds that payment coordination, reserves, early-submission incentives and the detailed design of LSM algorithms affect liquidity efficiency. It also finds that participant behavior can partly offset the intended effect of particular mechanism designs. [13]

Garratt's Shapley-value approach. Instead of simply maximizing settlement value, this work argues that welfare depends on participants' benefits and liquidity costs. It proposes determining both which payments settle and how liquidity costs are allocated, using Shapley-value ideas to improve participant acceptance of netting proposals. This is substantially more economically sophisticated than ClearPay and is best used as literature context rather than copied into your student implementation. [14]

Project Titus / auction-based LSM. The 2025 BIS work goes even further: rather than holding liquidity fixed and optimizing the payment subset, participants bid on their willingness to supply liquidity, with side payments compensating liquidity providers. Simulations calibrated to four RTGS systems reported an average auction success rate of 79%, with reported liquidity savings relative to conventional RTGS ranging from 2.6% to 12.2%. This mechanism requires preference elicitation and participant-level economic incentives, making it considerably more complex than ClearPay. [15]

Payment splitting and payment reordering are alternative approaches rather than direct competitors to ClearPay. Bank of England simulations found that splitting large payments can lower liquidity requirements, while Bank of Canada work explores optimizing payment order; a September 2026 Bank of Canada paper reports that simulated annealing can achieve results comparable with quantum optimization under the same time constraints in some queue-reordering experiments. These approaches suggest useful future extensions but change a different part of the problem than ClearPay's subset selection. [16]

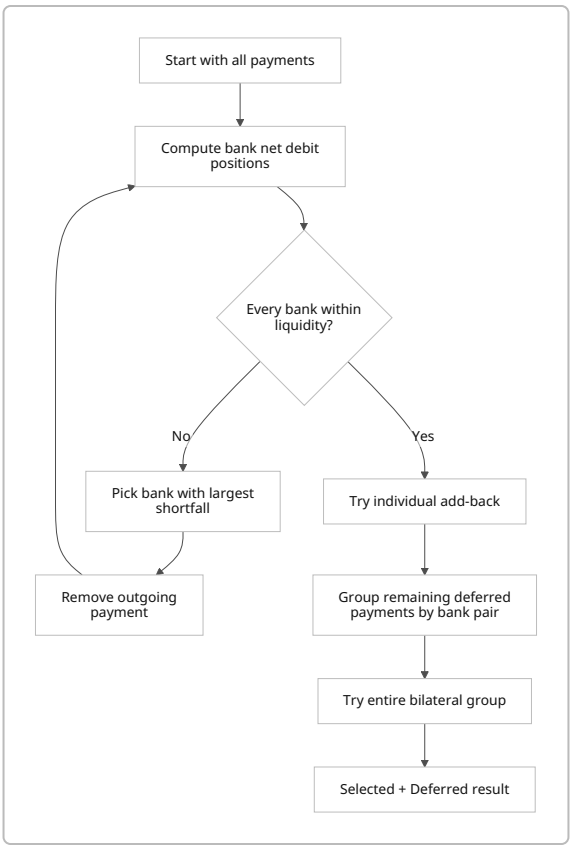
Direct comparison with ClearPay

Approach	Primary objective	Optimality	Computational character	Global coordination required	Serverless suitability	Free-tier/ local friendliness
Güntzer et al. BCP heuristic [4]	Maximize clearing volume subject to bank deposits	Heuristic; no global guarantee stated	Combinatorial clearing problem with heuristic approximation	Central view of obligations and deposits	**High** for small batches	**High** locally/ Lambda
Shafransky–Doudkin [5]	Maximize clearing under limited funds	Heuristic	Hard discrete optimization; fast heuristic proposed	Centralized	**High** for heuristic	**High**
Bech–Soramäki gridlock resolver [6]	Reduce queued/ gridlocked RTGS payments	Not presented as global value optimum	Repeated queue/gridlock processing	Central queue state	**Medium–High**	**High** for simplified simulation
CHAPS LSM [9]	Reduce liquidity needed for non-urgent queued payments	Operational matching mechanism, not published as raw-value optimum	Bilateral/group simultaneous offsetting	Strong centralized coordination	Conceptually **Medium**	Full mechanism unrealistic; simplified model **High**
TARGET2/T2 LSM [17]	Reduce liquidity use/ queueing while respecting priorities and reservations	Multiple operational optimization procedures	Event/time-driven queue algorithms	Strong centralized queue/ liquidity view	**Medium**	Simplified subset feasible
Lynx/gridlock mechanism [12]	Improve liquidity pooling and coordinated settlement	Mechanism outcome depends on queue behavior	Dynamic queue + participant behavior	Centralized infrastructure plus strategic banks	**Medium**	Simulation **High**

Approach	Primary objective	Optimality	Computational character	Global coordination required	Serverless suitability	Free-tier/ local friendliness
Shapley-value LSM [14]	Maximize system welfare, allocate liquidity cost	Welfare-oriented mechanism	Combinatorial optimization + cost allocation	High; participant costs/ preferences	**Low–Medium**	Best kept local
Project Titus auction LSM [15]	Coordinate willingness to provide liquidity and settle payment sets	Mechanism/ incentive objective, not simply maximum raw value	Auction + bids + side payments + simulation	**Very high**	**Low** for a student MVP	Local research extension only
Exact binary MILP	Maximize selected payment value subject to liquidity	Exact **if solver proves optimality**	NP-hard worst case; solver-dependent	Central snapshot	Local **High**, Lambda **Medium/ Low**	Excellent benchmark locally
Current ClearPay removal heuristic	Maximize feasible underlying settled value	**No guarantee**; supplied tests show 3.97% mean gap on 20 small cases	Current implementation is roughly quadratic in batch size because feasibility is repeatedly recomputed	One frozen queue/ liquidity snapshot	**High**	**High**
Recommended bounded group-repair ClearPay	Maximize value/utility with bounded pair and cycle recovery	Heuristic; bounded exact repair inside small groups	Predictable via explicit group/ search limits	One snapshot	**Very high**	**Very high**

The comparison leads to an important conclusion: ClearPay is closest intellectually to the **Bank Clearing Problem plus gridlock-resolution/LSM literature**, not to ordinary cloud payment APIs. The AWS work gives you an engineering contribution; the group-aware optimization gives you an algorithmic contribution.

Your present algorithm can be represented as:



That is reasonable, but the current **bilateral group stage is coarse**: every deferred payment between a bank pair is restored together or not at all. A subset of that pair could be highly valuable and feasible even when the full pair-group is not. That is the clearest algorithmic weakness to improve first.

I also found a small implementation issue in the supplied `liquidity.py`: the code documents lower numeric `priority` as higher business priority, but in one equal-amount removal tie the current `min(... priority ...)` selection can choose the **higher-priority payment for removal**. There is an unused `_removal_key` that appears intended to prefer low-priority removals. Fix this before presenting priority behavior as an evaluated feature.

Algorithmic improvements that are worthwhile

The strongest path is **not** to reproduce TARGET, CHAPS or Project Titus. Those systems have central-bank infrastructure, real collateral, participant incentives and large operational teams. ClearPay should instead implement one sophisticated but bounded resolver and evaluate it properly. Industry evidence itself shows that LSM design involves trade-offs among liquidity, delay, risk and incentives rather than one universally dominant algorithm. [18]

Recommended core improvement: bounded subset group repair

Instead of:

> “Take all deferred A↔B payments and add the whole group if feasible.”

do:

> “For each important bank pair, search a bounded number of deferred payments for the best feasible subset.”

For example, keep at most `M = 10–12` candidate payments per bank pair. A brute-force search of twelve binary choices has only $2^{12} = 4096$ subsets, which is tiny for a local/student workload. Because `M` is a configuration limit rather than total queue length, runtime remains predictable.

```
BOUNDED_PAIR_REPAIR(selected, deferred, liquidity, M):

    groups = group deferred payments by unordered bank pair

    for group in groups ordered by descending candidate value:

        candidates = top M payments
                     by priority + age + amount

        best_subset = empty

        for each subset of candidates:
            if selected + subset is feasible:
                if utility(subset) > utility(best_subset):
                    best_subset = subset

        selected += best_subset
        deferred -= best_subset

    return selected, deferred
```

With `G` bank-pair groups and maximum candidate group size `M`, the bounded enumeration cost is approximately:

$$O(G\,2^M\,C_f)$$

where C_f is the feasibility-check cost. With incremental bank positions, C_f for a bilateral candidate can be reduced to a small constant number of affected bank updates rather than recomputing the entire batch.

The general clearing problem is combinatorial, which is why the literature repeatedly uses heuristics or specialized optimizers. [19]

Next improvement: bounded three-bank cycle recovery

Bilateral matching does not capture all offsetting opportunities. Real LSMs can use multilateral offsetting, and TARGET2's documentation explicitly discusses bilateral and multilateral offsetting. [20]

You can therefore add a **strictly bounded three-bank cycle stage**:

```
CYCLE_REPAIR(selected, deferred, liquidity, M, K):

    build directed bank graph from deferred payments

    find cycles A -> B -> C -> A

    rank cycles by:
        total candidate utility
        divided by additional net liquidity required

    keep only top K cycles

    for each retained cycle:

        take at most M candidate payments per edge

        search bounded combinations

        if best combination is feasible:
            restore it

    return updated selection
```

With B banks, naive triangle enumeration is $O(B^3)$. In your existing eight-bank experiments, this is tiny. The payment combinations should nevertheless be bounded explicitly—for example, at most two or three candidates per directed edge and only the top `K` cycles.

This is far more defensible than saying “our bilateral group method is new.” You can say you implemented a **bounded multi-level gridlock resolver** that searches individuals, bilateral structures and selected three-bank cycles under an explicit computation budget.

Add priority and aging as a separate objective

Real systems do not optimize payment value alone. T2 includes explicit payment priorities and liquidity reservations, while payment-system research emphasizes the trade-off between liquidity and delay. [21]

Instead of pretending one metric covers everything, calculate two scores:

$$V(S) = \sum_{p \in S} amount_p$$

and

$$U(S) = \sum_{p \in S} amount_p \left(1 + \alpha\, priorityScore_p + \beta\, ageScore_p\right)$$

Then report **raw settled value** and **priority-weighted utility** separately.

A possible policy could be:

```
HIGH   priority_score = 2
NORMAL priority_score = 1
LOW    priority_score = 0

age_score =
    min(deferral_count / AGE_CAP, 1)
```

Do not choose α and β secretly to make your proposed method win. Define them before experiments and include a sensitivity test.

This also gives you a better answer to starvation: you cannot guarantee that a payment settles if its sender never has enough liquidity, but aging can increase its selection priority in subsequent feasible windows.

Use look-ahead removal rather than purely amount-based removal

Your current removal rule makes a local choice based primarily on payment size. Removing an outgoing payment helps its sender but removes an incoming credit from its receiver, potentially making the receiver underfunded.

A bounded look-ahead version is stronger:

```
LOOKAHEAD_REMOVAL(selected, liquidity, K):

    while selected is infeasible:

        b = bank with largest shortfall

        candidates =
            top K outgoing payments of b
            that could reduce the shortfall

        for p in candidates:
            trial = selected - {p}
```

```
            score[p] = (
                total_shortfall(trial),
                lost_priority_utility(p),
                lost_value(p)
            )

        remove payment with lexicographically lowest score
```

With fixed `K`, this retains predictable performance. It also directly addresses the reviewer's criticism that a poor removal decision can later make add-back look artificially impressive.

Add randomized restarts as a local-only improvement

Because heuristics depend on tie-breaking and local choices, run the algorithm several times with controlled alternative tie-breaks:

```
best = deterministic_solution

for seed in [0, 1, ... R-1]:
    solution = heuristic(
        randomized_tie_breaks=True,
        seed=seed
    )

    if solution.utility > best.utility:
        best = solution

return best
```

Current complexity becomes approximately R times the base heuristic, but this is entirely reasonable for local benchmarking with `R = 8` or `16`.

The idea of using bounded, time-constrained stochastic optimization is consistent with modern central-bank work on payment ordering: recent Bank of Canada research compares simulated annealing and quantum optimization under matched time budgets and finds simulated annealing competitive in the tested queue-reordering setting. [22]

Use MILP as an oracle and optional neighborhood repair, not as the cloud algorithm

Your friend's exact formulation is good:

$$\max \sum_p v_p x_p$$

subject to, for every bank b,

$$\sum_{p:from(p)=b} v_p x_p - \sum_{p:to(p)=b} v_p x_p \le A_b$$

with:

$$x_p \in \{0,1\}.$$

This is a binary mixed-integer program. SciPy documents `milp` specifically for binary selection/knapsack-style problems, which makes it suitable as a reference implementation for your experiments. [23]

A particularly useful extension is **neighborhood MILP repair**:

```
heuristic_solution = bounded_group_heuristic(...)

candidate_neighborhood =
      top K deferred payments
    + bottom J currently-selected payments

fix all payments outside the neighborhood

solve MILP only for candidate_neighborhood
with a short solver time budget

if solver returns a verified feasible improvement:
    adopt it
else:
    keep heuristic_solution
```

This lets you study questions such as:

> "How much additional value can a 500 ms optimization repair obtain after our fast heuristic?"

You do not need this inside Lambda. Run it locally. The local MILP is your **quality oracle**, while the lightweight pure-Python heuristic is your deployable production-style algorithm.

A good final hierarchy is therefore:

Level	Algorithm	Role
Baseline	All-or-nothing	Weak reference
Baseline	Cumulative greedy	Sequential reference
Proposed A	Removal + single add-back	Existing
Proposed B	Bounded bilateral subset repair	**New implementation target**

Level	Algorithm	Role
Proposed C	Bounded pair + 3-cycle repair	**Strongest project version**
Optional	Randomized restarts	Quality improvement
Reference	Exact/time-limited MILP	Local optimality benchmark

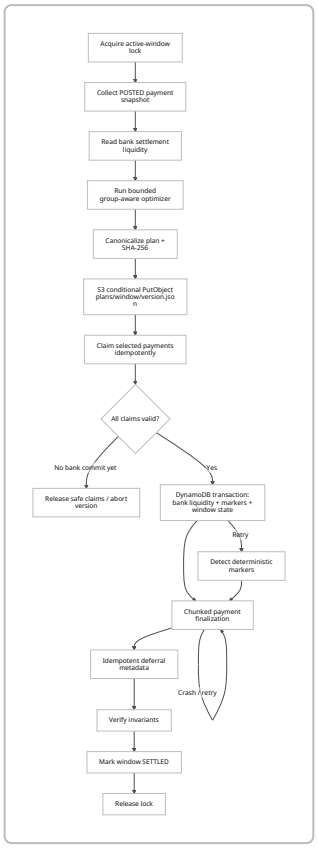
That is enough algorithmic depth for a strong project.

AWS serverless implementation and cost controls

Your AWS architecture is valuable because the academic literature mostly focuses on clearing algorithms and economic mechanism design; ClearPay can demonstrate how such a resolver is actually executed **reliably under retries, duplicated messages and partial failures**.

AWS also requires this because SQS/Lambda delivery is retry-oriented rather than inherently “exactly once.” AWS recommends partial batch responses so successfully processed records are not unnecessarily retried, and recommends an SQS visibility timeout at least six times the Lambda timeout; it also recommends a DLQ redrive `maxReceiveCount` of at least five for Lambda-backed queues. [24]

Recommended final AWS architecture



Freeze the plan before claiming payments. Otherwise, a crash after `POSTED → IN_WINDOW` but before the plan is persisted can leave payments attached to a settlement execution that has no durable definition. S3 supports conditional object creation using `If-None-Match: *`, which prevents an existing object key from being silently overwritten. [25]

A plan should contain at least:

```
{
  "window_id": "2026-10-21T14:00:00Z",
  "plan_version": 1,
  "algorithm": "bounded_cycle_repair_v1",
  "payment_snapshot_hash": "...",
  "liquidity_snapshot": {},
  "selected_ids": [],
  "deferred_ids": [],
  "bank_net_movements": {},
  "objective_metrics": {},
  "created_at": "...",
  "plan_hash": "..."
}
```

Use canonical field ordering when calculating `plan_hash`.

Claims should be conditional. A selected payment can be claimed only when it is still `POSTED`, or when it is already `IN_WINDOW` for the same deterministic window/version. A payment claimed by another window must not be silently reused.

Conceptually:

```
POSTED
  -> IN_WINDOW(window_id, plan_version)
  -> SETTLED
```

Retrying the same claim is success; conflicting ownership is failure.

Keep the payment ledger and settlement liquidity conceptually separate. Your posting stage already records the financial obligation. Settlement should operate on a separate attribute such as:

```
available_liquidity_paise
```

rather than applying the original debit/credit again.

For minimum infrastructure change, that attribute can live on your existing bank/account balance item if the schema permits it; you do not necessarily need a fifth DynamoDB table.

Commit bank liquidity before marking payments settled. Otherwise a crash can produce payments that say `SETTLED` although the associated bank-level settlement movement never occurred.

For a small number of banks, use one `TransactWriteItems` operation containing:

```
Update Bank A liquidity
Put marker window#version#A

Update Bank B liquidity
Put marker window#version#B

...

Update settlement-window commit state
```

DynamoDB transactions support up to 100 transaction actions; under a two-actions-per-bank plus one-window-update design, the theoretical action-count ceiling is therefore approximately 49 banks for one atomic transaction. Your eight-bank student demonstration is comfortably below that design limit. [26]

Do **not** rely solely on DynamoDB's `ClientRequestToken` for long-term idempotency. AWS documents that the token's idempotency window lasts only ten minutes. Deterministic marker items and conditional states must therefore provide the durable guarantee. [27]

A payer update should also guard the live liquidity value:

```
Condition:
available_liquidity_paise >= :required
```

because liquidity may have changed between optimization and commit. A top-up is safe; unexpected consumption must prevent the stale plan from overdrawing the bank.

Finalize payments in chunks. Do not try to settle thousands of payment records in the same bank transaction. Once the bank-level commit has succeeded, process payments in bounded chunks such as 25–50 records and persist a cursor/checkpoint on the settlement record.

Each update should require:

```
status = IN_WINDOW
AND window_id = :w
AND plan_version = :v
```

If a retry sees the payment already `SETTLED` by the same window/version, treat it as success rather than an error.

Deferral counters also need idempotency. Avoid retrying:

```
ADD deferral_count 1
```

without a guard.

Instead store the intended next value along with:

```
last_deferred_window = window_id
```

and update only if that window has not already recorded a deferral.

Step Functions design

A small Standard Workflow is enough:

```
AcquireLock
   ↓
CollectSnapshot
   ↓
Optimize
   ↓
PersistPlan
   ↓
ClaimSelected
   ↓
CommitLiquidity
   ↓
FinalizeChunk ──┐
   ↑            │ more
   └────────────┘
   ↓
RecordDeferrals
   ↓
Verify
```

```
   ↓
CompleteWindow
```

Avoid one Step Functions state transition per payment. A Lambda should finalize a **chunk** of payments and return the next cursor.

AWS currently lists **4,000 free Standard Workflow state transitions per month**, and retries count toward transitions, so a compact manually triggered demo workflow is much friendlier than an hourly schedule continuously processing empty windows. [28]

Free-tier strategy

The safest project architecture is:

Activity	Where to run
10,000-payment experiments	Local computer
MILP	Local computer
Randomized restarts	Local computer
Large parameter sweep	Local computer
Streamlit dashboard	Local computer
Small real payment ingestion demo	AWS
4–8 bank settlement demonstration	AWS
Crash/retry demonstration	AWS, a handful of runs
Benchmark charts	Generated locally
Athena/QuickSight	**Skip unless required**

DynamoDB's currently published service free allowance for provisioned Standard tables includes 25 WCUs, 25 RCUs and 25 GB of storage; that allowance is specifically based on **provisioned capacity**, so on-demand tables should not be described as covered by those provisioned WCU/RCU allowances. [29]

SQS currently includes one million free requests each month, and Step Functions lists 4,000 free Standard state transitions monthly. These allowances make a small student demonstration inexpensive, but they do not prove that your entire account will have a ₹0 bill. [30]

AWS changed its Free Tier structure in July 2025. New users can choose a Free account plan with credits and a six-month eligibility period, while legacy-account rules differ; the user's actual account plan is therefore an **unspecified dependency** until checked in Billing and Cost Management. AWS explicitly provides Free Tier usage monitoring for this purpose. [31]

Therefore write this in your report:

> **“ClearPay is designed to minimize AWS expenditure and to operate within applicable AWS Free Tier allowances or account credits for the demonstration workload. Large-scale experimental benchmarking is executed locally. Zero cloud cost is not assumed and depends on account eligibility and actual usage.”**

Do not write “the project runs completely free on AWS” unless your billing console actually demonstrates that.

Evaluation, benchmarking, and dashboard integration

Your existing friend's benchmark is a useful start but should be upgraded before the final report.

The current supplied main benchmark uses **200 payments, eight banks, ten seeds and liquidity levels from 10% to 120% of gross outgoing payment value**. At selected levels, the supplied results are:

Liquidity setting	All-or-nothing	Cumulative greedy	Removal heuristic
10%	0.0%	66.6%	88.0%
20%	0.0%	87.9%	94.6%
30%	10.0%	94.9%	97.6%
40%	40.0%	98.1%	99.0%
50%	80.0%	99.5%	99.6%
60%	90.0%	99.7%	99.9%
70%	100.0%	99.9%	100.0%

These are **synthetic local benchmark results supplied with the project**, not real banking-system performance figures.

The fact that greedy can settle **less** than all-or-nothing in some cases is not necessarily a bug: greedy evaluates sequentially and may reject individually infeasible obligations even though the complete batch becomes feasible through simultaneous offsetting. That is exactly why group-aware clearing matters.

However, you need stronger experiments than one random generator.

Local benchmark matrix

Run at least these workload families:

Dataset	Banks	Payments	Why
Tiny exhaustive	3–5	10–20	Brute-force correctness
MILP benchmark	4–12	25–500	Optimality comparison where solver permits

Dataset	Banks	Payments	Why
Uniform random	8/16/25	200–10,000	Scaling
Hub-spoke	8/16/25	200–10,000	Concentrated network
Community/block	12/24	500–5,000	Clustered payment flow
Bilateral-heavy	8/16	200–2,000	Tests pair repair
Cycle-heavy	6/12	200–2,000	Tests three-bank recovery
Priority stress	8	500	High/normal/low fairness
Multi-window	8	200/window	Deferred-payment aging/top-ups

Use predetermined seeds such as:

```
2026
2027
2028
...
2055
```

Thirty seeds gives substantially better stability than showing one hand-picked example.

For every solution, independently verify using integer arithmetic:

$$netDebit_b(S) \leq liquidity_b$$

for every bank.

Metrics

Do not collapse the project into one “efficiency” percentage.

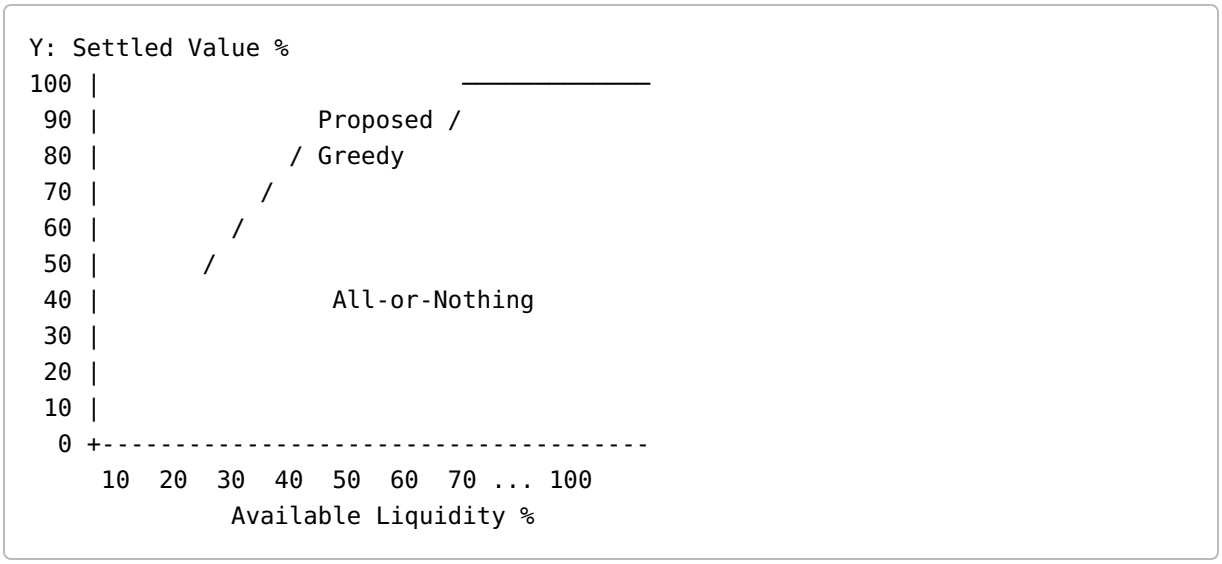
Report:

Metric	Meaning
Settled value %	Main throughput/value objective
Settled count %	Number of payments completing
Net liquidity consumed	Absolute liquidity usage
Liquidity / 1,000 settled	Efficiency metric
Priority-weighted utility	Policy-sensitive performance

Metric	Meaning
High-priority settlement %	Priority behavior
Mean/p95 deferral windows	Fairness/delay
Maximum deferral age	Starvation signal
Execution time	Computational cost
MILP optimality gap	Heuristic quality
Number of group/cycle repairs	Explainability
Duplicate bank effects after failures	Reliability
Reconciliation precision/recall	Accounting quality

A key plot should be:

Settled payment value versus available liquidity



The actual graph must come from measured results, not this schematic.

The other most useful final-report plots are:

Plot	What it answers
Settled value vs liquidity	Does the algorithm help under scarcity?
Settled count vs liquidity	Does value improvement also help payment count?
Optimality-gap boxplot	How close is the heuristic to MILP?
Runtime vs batch size, log scale	Does it scale?

Plot	What it answers
Pair-only vs pair+cycle quality/runtime	Is extra group search worth it?
Priority settlement vs liquidity	Does priority handling work?
Deferral-age distribution	Are payments starving?
Failure point vs duplicate financial effects	Does recovery work?

MILP comparison

For tiny instances, enumerate every subset and prove the answer exactly.

For medium instances, use SciPy MILP. SciPy's optimization documentation explicitly demonstrates binary integer selection problems using `milp`; solver results should then be rechecked with integer-paise arithmetic because the optimization backend operates numerically. [23]

Report:

$$gap = \frac{V_{\text{best reference}} - V_{\text{heuristic}}}{V_{\text{best reference}}} \times 100\%.$$

Do not call a time-limited solver result "the true optimum" unless optimality is actually established. Call it **best MILP incumbent** and report the solver gap/bound when available.

AWS failure-injection tests

These tests distinguish ClearPay from an ordinary algorithm assignment:

Failure test	Required outcome
Same API request 5×	One payment business effect
SQS duplicate delivery	One ledger-posting effect
One message fails inside batch	Successful neighbors not unnecessarily retried
Crash immediately after S3 plan	Same plan reused
Crash halfway through claims	Idempotent claiming resumes
Crash before bank commit	No settlement liquidity moves
Crash immediately after bank commit	Bank movement not repeated
Crash during payment finalization	Remaining chunks resume
Settlement started twice	Active-window lock prevents competing commit
Same deferral step retries	`deferral_count` increments once

Failure test	Required outcome
Liquidity changed after planning	Bank commit condition rejects stale underfunded plan
Poison queue message	Eventually reaches DLQ

The SQS duplicate/retry tests are important because AWS explicitly documents that a failed Lambda batch ordinarily causes records to become visible again unless partial batch responses are configured. [32]

Invariants

After every test assert:

```
No SETTLED payment belongs to two windows.

No bank settlement marker is applied twice.

No payer's available settlement liquidity is negative.

Sum of bank settlement liquidity is conserved
unless a top-up or withdrawal was deliberately applied.

Every SETTLED payment appears in the frozen plan.

Every frozen plan hash is stable across retries.

Every deferred payment remains eligible for a later window.

deferral_count increases at most once per window.
```

These invariants are stronger evidence than terminal screenshots saying “PASSED.”

Minimal integration with your existing SAM project

You do **not** need to rewrite the project.

Your friend's code uses:

```
from
to
amount_paise
```

while your existing API uses:

```
from_account
to_account
amount_paise
```

Add a small adapter:

```python
def to_liquidity_payment(payment: dict) -> dict:
    return {
        "payment_id": payment["payment_id"],
        "from": payment["from_account"],
        "to": payment["to_account"],
        "amount_paise": payment["amount_paise"],
        "priority": payment.get("priority", 1),
        "created_at": payment.get("created_at", ""),
    }
```

Then integrate in this order:

Priority	Change	AWS impact
P0	Create integration Git branch	None
P0	Copy `src/common/liquidity.py`	None
P0	Copy liquidity tests and benchmark scripts	None
P0	Fix priority/removal bug	None
P0	Make all existing + new tests pass	None
P1	Add `available_liquidity_paise`	DynamoDB data only
P1	Modify settlement `net`/optimizer Lambda to invoke heuristic	Lambda update
P1	Add deterministic window/version schema	DynamoDB
P1	Freeze optimization plan in existing S3 bucket	S3
P1	Add payment claiming/finalization guards	Lambda/DynamoDB
P1	Add active-window lock	DynamoDB
P1	Update Step Functions workflow	Step Functions
P2	Add bounded pair repair	Code only
P2	Add bounded three-cycle repair	Code only
P2	Run benchmark suite	Local
P3	Build Streamlit dashboard	Local

Priority	Change	AWS impact
P3	Deploy small final AWS update	SAM/CloudFormation
P3	Run final failure demo	AWS

Keep `scipy`, MILP and large benchmark dependencies **out of the Lambda deployment package** initially. The deployable resolver can remain pure Python.

Dashboard

Your teacher should see the project rather than terminal text.

The recommended Streamlit dashboard has five screens:

Overview

```
Total Payments       1,250
Posted               1,173
Settled              1,041
Deferred               132
Settlement Value     ₹...
Current Window       ...
```

Bank Liquidity

Bank	Available liquidity	Net obligation	Position
BANK01	₹10,000	₹13,000	Short ₹3,000
BANK02	₹5,000	₹0	Receiver
BANK03	₹7,500	₹5,100	Feasible

Settlement Lab

Allow the user to choose:

```
All-or-Nothing
Greedy
Removal
Bounded Pair Repair
Pair + 3-Cycle Repair
```

Then show:

```
Selected payments
Deferred payments
Settled value %
Net liquidity required
Execution time
Reason for each deferral
```

Benchmark Results

Load the JSON generated by your local benchmark and display the measured graphs.

Reliability/Reconciliation

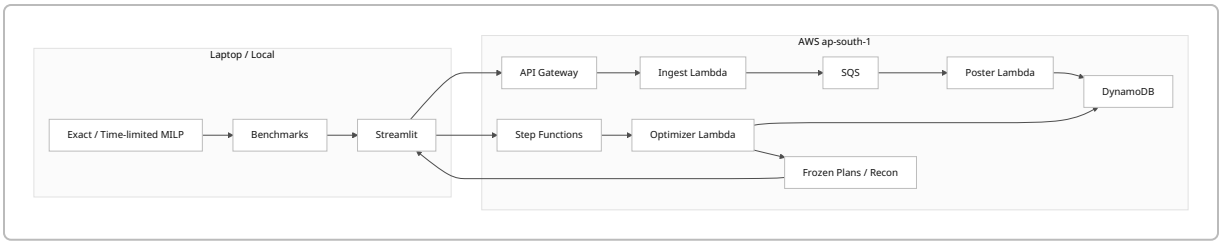
Show:

```
Injected failures
Retry count
Duplicate financial effects = 0

Missing statements
Duplicates
Amount mismatches
Matched records
```

The dashboard can run locally while calling your small AWS API for live payment operations. This avoids adding another continuously billed hosting layer.

The final implementation relationship is:



Prioritized implementation roadmap and novelty wording

The best final project is **not the most complicated possible algorithm**. It is the smallest architecture for which you can prove meaningful optimization and correct recovery.

Recommended implementation priority

Rank	Build	Why
Must	Integrate friend's current liquidity code into your repo	Establish one codebase
Must	Fix priority semantics and add stronger tests	Current behavior must be correct
Must	Add separate settlement liquidity	Makes problem real in AWS
Must	Add immutable plan + conditional claims	Removes crash ambiguity
Must	Atomic bank liquidity commit + deterministic markers	Core correctness
Must	Chunked finalization	Makes recovery realistic
Must	Streamlit settlement dashboard	Teacher-facing final output
High	Bounded bilateral subset repair	Best algorithmic improvement for effort
High	Priority aging	Adds realistic operational objective
High	MILP comparison	Proves heuristic quality
High	Failure-injection demonstration	Strong cloud/distributed-systems contribution
Medium	Bounded 3-cycle repair	Stronger group-aware resolver
Medium	Randomized restarts	Local quality improvement
Medium	Multi-window top-up demonstration	Excellent dashboard story
Low	Athena	Not needed
Low	QuickSight	Not needed
Do not add	Auction/side-payment mechanism	Far beyond required scope
Do not add	Quantum optimization	Adds complexity without helping the project

The strongest final title remains:

> **ClearPay: A Liquidity-Aware, Fault-Tolerant Payment Clearing and Settlement System Using AWS Serverless Architecture**

A more algorithm-focused alternative is:

ClearPay: Group-Aware Liquidity-Constrained Payment Settlement with Fault-Tolerant Serverless Execution

I prefer the first because it accurately covers your full system rather than making the project sound like only an optimization algorithm.

Recommended problem statement

Interbank payment systems must settle large numbers of payment obligations while participating institutions have limited settlement liquidity. Under constrained liquidity, processing payments independently or using an all-or-nothing batch policy may delay transactions that could otherwise settle through coordinated offsetting. At the same time, asynchronous cloud processing introduces duplicate delivery, retries and partial-failure risks that can create inconsistent financial states if settlement is not idempotent and recoverable.

ClearPay addresses these problems by implementing a cloud-based payment clearing and settlement system that identifies feasible subsets of payment obligations using a bounded group-aware liquidity optimization heuristic, defers temporarily infeasible payments for later settlement, and executes each frozen settlement plan through retry-safe AWS serverless workflows. The system also provides automated reconciliation and comparative benchmarking against all-or-nothing, cumulative greedy and exact mixed-integer optimization baselines.

That problem statement is much stronger than “cloud-based netting.”

Recommended novelty claim

Use:

“ClearPay's project-level contribution is a bounded group-aware liquidity-constrained settlement controller integrated with a deterministic, crash-recoverable AWS serverless settlement workflow. The resolver combines removal-based feasibility restoration, bounded bilateral/three-bank group repair, payment priority aging and multi-window reconsideration. Its quality is evaluated against simple baselines and an exact or time-limited MILP reference.”

Also acceptable:

“The novelty of the project lies in the integration and evaluation of group-aware liquidity optimization with fault-tolerant serverless settlement, rather than in claiming invention of liquidity-saving mechanisms themselves.”

Do **not** use:

“We invented liquidity-saving settlement.”

Do **not** use:

> “Bilateral group add-back is a genuine new algorithm.”

Do **not** use:

> “ClearPay is the first system to settle groups of offsetting payments.”

Do **not** use:

> “ClearPay is always liquidity-optimal.”

Those statements are contradicted or unsupported by the established LSM literature and, in the last case, by your own MILP benchmark. [33]

Limitations to state honestly

> ClearPay is a simulated payment-settlement prototype rather than a production banking system. Its transaction data and liquidity balances are synthetic; it does not connect to central-bank money, real RTGS networks or actual bank collateral. The settlement heuristic does not guarantee a globally optimal subset, and bounded pair/cycle search may miss larger multilateral combinations. The model assumes a centrally observable settlement queue and does not model strategic participant bidding, collateral prices or all real-time delay incentives. Large-scale benchmarks are executed locally, while AWS is used for a small serverless integration and recovery demonstration.

That limitation statement is a strength, not a weakness. Real-world research shows that participant incentives, delay costs, collateral and LSM design materially affect outcomes. [34]

SDG wording

Keep **SDG 9** as primary and **SDG 8** as secondary.

Your existing project maps:

SDG	ClearPay contribution
SDG 9.1	Reliability/resilience through deferral, recovery and fault-tolerant settlement
SDG 9.5	Technical innovation and measured payment-processing/settlement optimization
SDG 8.2	Indirect technological-productivity connection
SDG 8.10	Indirect connection through financial-infrastructure and reconciliation tooling

Do not say your prototype has demonstrated increased GDP, employment or financial inclusion.

Your friend's current SDG metrics can be shown as **technical proxy indicators**. For example, her supplied experiment reports +29.3 percentage points of settled value over the all-or-nothing baseline in its selected synthetic conditions, 100% recovery of the tested W1-deferred payments after its W2 liquidity top-up, and detection of all fourteen deliberately injected reconciliation discrepancies. Present those as simulated software results, not as measured UN SDG outcomes.

Unspecified details that should remain explicit: your final project timeline was not provided; the desired maximum AWS live batch size is unspecified; your intended maximum number of banks is unspecified; and your AWS account's current Free/Paid plan and remaining credits are unspecified.

Priority sources and exact search strategy

These are the sources I would put first in your literature review. The first six are enough for a solid proposal; the rest make the final report stronger.

Priority	Source	Why it matters	Exact Google Scholar/search query
1	Güntzer, Jungnickel & Leclerc, *Efficient algorithms for the clearing of interbank payments*, EJOR 1998 [4]	Foundational problem closest to ClearPay	`"Efficient algorithms for the clearing of interbank payments" Güntzer Jungnickel Leclerc`
2	Bech & Soramäki, *Gridlock resolution in interbank payment systems*, Bank of Finland 2001 [6]	Direct gridlock-resolution literature	`"Gridlock resolution in interbank payment systems" Bech Soramäki`
3	Shafransky & Doudkin, *An optimization algorithm for the clearing of interbank payments*, EJOR 2006 [5]	Computational difficulty + heuristic design	`"An optimization algorithm for the clearing of interbank payments" Shafransky Doudkin`
4	Galbiati & Soramäki, *Liquidity-saving mechanisms and bank behaviour*, BoE WP 400 [35]	Shows strategic limitations of LSMs	`"Liquidity-saving mechanisms and bank behaviour" Galbiati Soramäki`
5	Jurgilas & Martin, *Liquidity-saving mechanisms in collateral-based RTGS payment systems*, BoE WP 389 [36]	Matching/offsetting and liquidity incentives	`"Liquidity-saving mechanisms in collateral-based RTGS payment systems" Jurgilas Martin`

Priority	Source	Why it matters	Exact Google Scholar/search query
6	Davey & Gray, CHAPS LSM analysis [37]	Real deployed LSM and measured liquidity effect	`"How has the Liquidity Saving Mechanism reduced banks intraday liquidity costs in CHAPS"`
7	ECB, TARGET2 LSM analysis [38]	Real bilateral/ multilateral queue optimization	`site:ecb.europa.eu "Strengths of liquidity-saving mechanisms in TARGET2"`
8	ECB, current T2 description [39]	Current operational features	`site:ecb.europa.eu T2 "liquidity-saving" optimisation procedures`
9	BIS, *New developments in large-value payment systems* [40]	Historical industry context	`site:bis.org "New developments in large-value payment systems" liquidity`
10	BIS WP 1089, *Intraday liquidity around the world* [13]	Cross-country empirical evidence	`site:bis.org "Intraday liquidity around the world"`
11	Garratt, *An Application of Shapley Value Cost Allocation to Liquidity Savings Mechanisms* [14]	Goes beyond raw-value objectives	`"An Application of Shapley Value Cost Allocation to Liquidity Savings Mechanisms"`
12	BIS, *Auction-based liquidity saving mechanisms* / Project Titus [15]	State-of-the-art incentive-based direction	`site:bis.org "Auction-based liquidity saving mechanisms"`
13	Bank of Canada, *From LVTS to Lynx* [41]	Modern deployed LSM behavior	`"From LVTS to Lynx" liquidity saving mechanism`
14	Bank of Canada, payment reordering/quantum work [42]	Useful for stochastic/ time-budgeted optimization discussion	`"Liquidity Optimization in Gross Settlement Systems with Quantum Reordering" TARGET2`
15	Denbee & Norman, payment splitting [43]	Alternative liquidity-saving intervention	`"impact of payment splitting on liquidity requirements in RTGS"`

For Google Scholar, also use thematic queries rather than searching only your proposed title:

```
"payment gridlock resolution" interbank
```

```
"liquidity saving mechanism" payment systems netting algorithm
```

```
"bank clearing problem" liquidity heuristic
```

```
"payment queue optimization" RTGS
```

```
"partial settlement" interbank payments liquidity
```

```
"multilateral offsetting" payment system gridlock
```

```
"liquidity constrained" payment clearing optimization
```

```
"priority payments" liquidity saving mechanism RTGS
```

```
"payment settlement" mixed integer programming liquidity
```

```
"gridlock resolution" bilateral multilateral offsetting
```

```
"payment settlement" heuristic optimality liquidity
```

```
"liquidity saving mechanism" AWS serverless
```

The final query is useful for establishing that the **cloud-engineering integration** is less commonly discussed than the financial optimization problem itself; absence of results, however, is not proof of novelty.

The literature supports a clear final positioning:

> **ClearPay is a serverless prototype of a liquidity-saving payment settlement controller. It does not invent gridlock resolution; instead, it implements and extends established liquidity-constrained clearing ideas through bounded group repair, priority-aware multi-window settlement, exact benchmark comparison and deterministic failure-safe AWS execution.**

That positioning is technically accurate, defensible in a viva, directly connected to the code you and your friend have already written, and leaves a manageable implementation path: **merge the code, fix the**

heuristic, add bounded pair/cycle repair, wire it into the existing Step Functions settlement workflow, freeze/recover the plan safely, and expose the entire process through the Streamlit dashboard.

---

1 4 19 33 Efficient algorithms for the clearing of interbank payments - ScienceDirect
https://www.sciencedirect.com/science/article/pii/S0377221797002658?utm_source=chatgpt.com

2 9 37 How has the Liquidity Saving Mechanism reduced banks' intraday liquidity costs in CHAPS? | Bank of England
https://www.bankofengland.co.uk/quarterly-bulletin/2014/q2/how-has-the-liquidity-saving-mechanism-reduced-banks-intraday-liquidity-costs-in-chaps

3 15 Auction-based liquidity saving mechanisms
https://www.bis.org/publications/working-paper-1318-auction-based-liquidity-saving-mechanisms?utm_source=chatgpt.com

5 An optimization algorithm for the clearing of interbank payments - ScienceDirect
https://www.sciencedirect.com/science/article/pii/S037722170400582X?utm_source=chatgpt.com

6 11 Gridlock resolution in interbank payment systems
https://publications.bof.fi/items/6a2d656d-32dd-428e-a09d-b547a9a00d73

7 18 40 New developments in large-value payment systems
https://www.bis.org/publications/new-developments-large-value-payment-systems?utm_source=chatgpt.com

8 34 35 Liquidity-saving mechanisms and bank behaviour | Bank of England
https://www.bankofengland.co.uk/working-paper/2010/liquidity-saving-mechanisms-and-bank-behaviour

10 17 20 38 TARGET Annual Report 2020
https://www.ecb.europa.eu/press/targetar/html/ecb.targetar2020.el.html?utm_source=chatgpt.com

12 41 From LVTS to Lynx: Quantitative Assessment of Payment System Transition - Bank of Canada
https://www.bankofcanada.ca/2023/04/staff-working-paper-2023-24/?utm_source=chatgpt.com

13 Intraday liquidity around the world
https://www.bis.org/publications/working-paper-1089-intraday-liquidity-around-world?utm_source=chatgpt.com

14 An Application of Shapley Value Cost Allocation to Liquidity Savings Mechanisms - Bank of Canada
https://www.bankofcanada.ca/2019/07/staff-working-paper-2019-26/?utm_source=chatgpt.com

16 43 The impact of payment splitting on liquidity requirements in RTGS | Bank of England – the UK's central bank
https://www.bankofengland.co.uk/working-paper/2010/the-impact-of-payment-splitting-on-liquidity-requirements-in-rtgs?utm_source=chatgpt.com

21 TARGET Annual Report 2023
https://www.ecb.europa.eu/press/targetar/html/ecb.targetar2023.en.html?utm_source=chatgpt.com

22 Liquidity Optimization in Gross Settlement Systems with Quantum Reordering: Application to TARGET2∗ - Bank of Canada
https://www.bankofcanada.ca/2026/09/staff-working-paper-2026-32/?utm_source=chatgpt.com

23 Optimization (scipy.optimize) — SciPy v1.18.0 Manual
https://docs.scipy.org/doc/scipy/tutorial/optimize.html?utm_source=chatgpt.com

24 Creating and configuring an Amazon SQS event source mapping - AWS Lambda
https://docs.aws.amazon.com/lambda/latest/dg/services-sqs-configure.html?utm_source=chatgpt.com

25 How to prevent object overwrites with conditional writes - Amazon Simple Storage Service
https://docs.aws.amazon.com/AmazonS3/latest/userguide/conditional-writes.html?utm_source=chatgpt.com

26 Class: Aws::DynamoDB::Types::TransactWriteItemsInput — AWS SDK for Ruby V3
https://docs.aws.amazon.com/sdk-for-ruby/v3/api/Aws/DynamoDB/Types/TransactWriteItemsInput.html?utm_source=chatgpt.com

27 Amazon DynamoDB Transactions: How it works - Amazon DynamoDB
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/transaction-apis.html?utm_source=chatgpt.com

28 Preço do AWS Step Functions
https://aws.amazon.com/pt/step-functions/pricing/?utm_source=chatgpt.com

29 Preços do Amazon DynamoDB | Banco de dados de chave-valor NoSQL | Amazon Web Services
https://aws.amazon.com/pt/dynamodb/pricing/?utm_source=chatgpt.com

30 Amazon SQS Pricing
https://aws.amazon.com/sqs/pricing/?utm_source=chatgpt.com

31 AWS Free Tier now offers $200 in credits and 6-month free plan to explore AWS at no cost - AWS
https://aws.amazon.com/about-aws/whats-new/2025/07/aws-free-tier-credits-month-free-plan/?utm_source=chatgpt.com

32 Handling errors for an SQS event source in Lambda - AWS Lambda
https://docs.aws.amazon.com/en_en/lambda/latest/dg/services-sqs-errorhandling.html?utm_source=chatgpt.com

36 Liquidity-saving mechanisms in collateral-based RTGS payment systems | Bank of England
https://www.bankofengland.co.uk/working-paper/2010/liquidity-saving-mechanisms-in-collateral-based-rtgs-payment-systems

39 What is T2?
https://www.ecb.europa.eu/paym/target/t2/html/index.ga.html?utm_source=chatgpt.com

42 Improving the Efficiency of Payments Systems Using Quantum Computing - Bank of Canada
https://www.bankofcanada.ca/2022/12/staff-working-paper-2022-53/?utm_source=chatgpt.com